위상수학적 음악 분석 — Step 2

두 가지 음악 생성 알고리즘

본 장에서는 본 연구의 두 가지 음악 생성 알고리즘 — Algorithm 1 (확률적 샘플링) 과 Algorithm 2 (신경망 기반 시퀀스 모델) — 의 핵심 아이디어와 설계 의도를 설명한다.

표기 정의

본 장에서 사용할 표기를 다음과 같이 통일한다.

기호	의미	hibari 값
T	시간축 길이 (8분음표 단위)	$1{,}088$
N	고유 note 수 (pitch-duration 쌍)	23
C	발견된 전체 cycle 수	최대 48
K	선택된 cycle subset 크기 ($K \leq C$)	$\{10, 17, 48\}$
O	중첩행렬, $\{0,1\}$ 값의 $T \times K$ 행렬	—
L_t	시점 t에서 추출할 note 개수	보통 3 또는 4
$V(c)$	cycle c의 vertex(note label) 집합	원소 수 $4 \sim 6$
R	재샘플링 최대 시도 횟수	50
B	학습 미니배치 크기	32
E	학습 epoch 수	200
H	DL 모델의 hidden dimension	128

L_t에 대한 보충. L_t는 "시점 t에서 새로 발성할 note의 개수"이다. hibari의 경우 악기 1, 2의 chord height(한 시점의 동시 발음 수)를 따라 대체로 3 또는 4로 설정되며, 구체적으로 `[4, 4, 4, 3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3]`의 32개 패턴을 33번 반복한 길이 $1{,}056$의 수열을 사용한다 (총합 약 $3{,}700$). 이 패턴은 원곡의 평균 density에 맞춰 경험적으로 결정된 것이다.

B, E, H에 대한 보충. 본 연구의 hidden dimension $H = 128$, epoch 수 $E = 200$, batch size $B = 32$는 **엄밀한 grid search로 튜닝된 값이 아니라**, 소규모 시퀀스 데이터에 대한 일반적 관례(LSTM 논문, Transformer 구현 예제)에서 차용한 출발점 값이다. 모든 실험을 동일한 하이퍼파라미터에서 수행함으로써 세 모델(FC / LSTM / Transformer)의 구조적 차이를 공정하게 비교하는 것을 우선했다. 향후 연구에서 이 값들에 대한 체계적 튜닝 여지가 남아있다.

2.10 Algorithm 1 — 확률적 샘플링 기반 음악 생성

알고리즘 개요

Algorithm 1은 중첩행렬의 ON/OFF 패턴을 직접 참조하여, 각 시점에서 활성화된 cycle들이 공통으로 포함하는 note pool로부터 확률적으로 음을 추출하는 규칙 기반 알고리즘이다. 신경망 학습 없이 즉시 생성이 가능하며, 중첩행렬이 곧 "구조적 seed" 역할을 한다.

핵심 아이디어 (3가지 규칙)

규칙 1 — 시점 t에서 활성 cycle이 있는 경우, 즉

$$\sum_{c=1}^{K} O[t,c] > 0$$

일 때, 활성화되어 있는 모든 cycle들의 vertex 집합의 교집합

$$I(t) = \bigcap_{c\,:\,O[t,c]=1} V(c)$$

에서 note 하나를 **균등 추출**한다. 여기서 "균등 추출"이란 집합 $I(t)$의 모든 원소가 동일한 확률 $1/|I(t)|$로 선택된다는 의미이다 (이산균등분포). 만약 교집합이 공집합이면 ($I(t)=\emptyset$), 전체 note pool P에서 균등 추출한다. 이 규칙은 "여러 cycle이 동시에 살아 있을 때, 그 cycle들이 모두 공유하는 note는 음악적으로도 가장 핵심적인 음"이라는 가정을 반영한다.

규칙 2 — 시점 t에서 활성 cycle이 없는 경우, 즉

$$\sum_{c=1}^{K} O[t,c] = 0$$

일 때, 인접 시점 $t-1, t+1$에서 활성화된 cycle들의 vertex의 합집합

$$A(t) = \bigcup_{c\,:\,O[t-1,c]=1} V(c) \cup \bigcup_{c\,:\,O[t+1,c]=1} V(c)$$

을 계산한 뒤, 전체 note pool에서 이 합집합을 제외한 영역 $P \setminus A(t)$에서 균등 추출한다.

이 규칙은 다음과 같은 의미를 가진다. 활성 cycle이 없는 시점에서도 음악은 흘러가야 하므로 음을 하나 골라야 하는데, 만약 인접 시점의 cycle 멤버 노트를 그대로 골라 버리면, 청자가 들었을 때 마치 그 cycle이 시점 t에도 살아 있는 것처럼 들리게 된다. 즉, 원래 분석상으로는 죽어 있어야 할 위상 구조가 인위적으로 살아있는 것처럼 "번지는" 현상이 생긴다. 이를 막기 위해 인접 cycle의 vertex를 의도적으로 회피하여, "활성 cycle 없음"이라는 정보가 청각적으로도 그대로 보존되도록 한다.

규칙 3 — 중복 onset 방지. 같은 시점 t에서 동일한 (pitch, duration) 쌍이 두 번 추출되지 않도록 `onset_checker`로 검사하며, 충돌이 발생하면 최대 R회까지 재샘플링한다. R회 모두 실패하면 그 시점의 해당 note 자리는 비워둔다.

출력

알고리즘은 (start, pitch, end) 형태의 음표 리스트 G를 출력하며, 이를 MusicXML로 직렬화하면 곧바로 악보 및 오디오로 재생할 수 있다.

2.11 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

알고리즘 개요

Algorithm 2는 중첩행렬을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑

$$f_\theta : \{0,1\}^{T\times C} \longrightarrow \mathbb{R}^{T\times N}$$

을 학습한다 (FC 모델은 시점별 독립이므로 $\{0,1\}^C \to \mathbb{R}^N$). 학습된 모델은 학습 시 보지 못한 cycle subset이나 노이즈가 섞인 중첩행렬에 대해서도 원곡과 닮은 note 시퀀스를 출력하도록 기대된다.

"위상 구조 보존"의 의미. 엄밀히 말하면 DL 모델은 Algorithm 1처럼 "교집합 규칙"으로 위상 구조를 직접 강제하지는 않는다. 대신 Subset Augmentation(아래 설명)을 통해 $K \in \{10, 15, 20, 30, 46\}$과 같은 다양한 크기의

subset에 대해서도 같은 원곡 y를 복원하도록 학습한다. 이 과정에서 모델은 "서로 다른 cycle subset이 같은 음악을 유도할 때, 그 공통적인 구조적 특성"을 잠재 표현으로 내부화한다. 따라서 학습 시 구체적으로 보지 못한 subset(예: $K=12$)에 대해서도, 모델이 학습한 잠재 표현이 충분히 일반화되어 있다면 합리적 출력이 가능하다. 본 연구의 실험에서는 이러한 일반화가 실제로 관측되었으나, 이는 "위상 구조 자체의 수학적 보존"이라기보다 "학습된 매핑의 통계적 일반화"에 가깝다.

모델 아키텍처 비교

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

모델	입력 형태	시간 정보 처리 방식	파라미터 수 (대략)
FC	(B, C)	시점 독립	4×10^4
LSTM (2층)	(B, T, C)	순방향 hidden state	2×10^5
Transformer (2층, 4-head)	(B, T, C)	self-attention	4×10^5

"2층 LSTM"의 의미. LSTM의 "layer"는 LSTM 셀을 수직으로 쌓은 개수이다. 1층 LSTM은 한 시퀀스를 한 번 통과시켜 hidden state를 만들고, 2층 LSTM은 1층의 출력을 다시 또 다른 LSTM 셀에 통과시켜 더 깊은 시간 표현을 만든다. 본 연구는 2층을 사용하여 간단한 시간적 상관관계(1층)와 더 복잡한 조합(2층)을 모두 포착하려 했다.

"2층, 4-head Transformer"의 의미. Transformer의 "layer"는 self-attention + feedforward 블록을 수직으로 쌓은 개수이며 본 연구는 2층을 사용했다. "head"는 self-attention을 병렬로 여러 개 실행하는 개수로, 4개의 head가 동시에 시퀀스의 서로 다른 관계(예: 짧은 거리 / 긴 거리 / 주기적 / 급변)를 독립적으로 학습한 뒤 결과를 합친다.

"시점 독립"의 의미 (FC). FC 모델은 시점 t의 위상 벡터 $O[t,:]$를 입력으로 받아 시점 t의 note 벡터 $y[t,:]$를 출력한다. 즉 시점 t의 출력은 이전 시점 $t-1, t-2, \ldots$나 이후 시점 $t+1, \ldots$을 전혀 참조하지 않는다. 각 시점을 "독립적으로" 처리한다는 뜻이다. 이는 가장 단순한 기준 모델이다.

"순방향 hidden state"의 의미 (LSTM). LSTM은 시점 t의 출력을 만들 때, 내부에 유지하는 hidden state h_{t-1}을 참조한다. h_{t-1}은 시점 $1, 2, \ldots, t-1$까지의 정보가 누적된 벡터이다. 따라서 "왼쪽에서 오른쪽으로 흐르는 시간 정보"를 사용한다. 미래 시점은 보지 못한다.

"self-attention"의 의미 (Transformer). Self-attention은 시점 t의 출력을 만들 때, 시퀀스의 **모든** 시점 $1, \ldots, T$에 대한 "주목도 점수"를 계산하여, 각 시점의 벡터를 가중합한다. LSTM과 달리 미래 시점도 함께 본다(bidirectional). 따라서 "시점 t의 note를 결정할 때 곡 전체의 문맥을 고려한다"는 해석이 가능하다.

학습 데이터 구성과 증강

원본 학습 쌍은 $X \in \{0,1\}^{T \times C}$, $y \in \{0,1\}^{T \times N}$이다. 여기서 X는 중첩행렬이고 y는 같은 시간축의 multi-hot note 행렬(시점 t에 활성인 note를 1로 표시)이다. 본 연구는 세 가지 증강 전략을 적용하여 학습 데이터를 약 $7 \sim 10$배 늘린다.

(1) Subset Augmentation. $K \in \{10, 15, 20, 30\}$의 cycle subset에 대한 중첩행렬을 생성하여, 동일한 정답 y에 매핑한다. 이를 통해 모델은 "불완전한 위상 정보로부터도 원곡을 복원하는" 강건한 표현을 학습한다.

(2) Circular Shift. 시간축을 회전하는 증강이며, X와 y를 **동일한 양만큼** 함께 회전한다. 즉

$$X' = \mathrm{roll}(X, s, \mathrm{axis}=0), \qquad y' = \mathrm{roll}(y, s, \mathrm{axis}=0)$$

로 처리한다 (여기서 s는 같은 난수). 여기서 `roll` 함수는 파이썬 고유 함수가 아니라 수치 연산 라이브러리 `numpy`에서 제공하는 함수(`numpy.roll`)이며, 배열을 지정한 축으로 원형 이동(끝에서 밀려난 원소가 앞으로 돌아옴)시키는 역할을 한다. 이 증강은 시점 t의 위상 정보와 시점 t의 note를 시점 $t+s$로 똑같이 옮기므로, 모델이 학습해야 할 매핑 자체는 변하지 않은 채 시작점만 달라진다. 만약 X에만 회전을 적용하면 X와 y의 시간축이 어긋나 학습 데이터가 망가지므로, 두 행렬을 항상 함께 회전해야 한다.

(3) Noise Injection. X에 확률 $p=0.03$으로 bit flip을 가한다 (y는 그대로). overfitting을 막고 정규화 효과를 얻기 위함이다.

학습 손실 함수

각 시점에서 여러 note가 동시에 활성화될 수 있으므로(multi-label 문제), 2.8절에서 정의한 binary cross-entropy 손실을 사용한다. PyTorch의 `BCEWithLogitsLoss`는 sigmoid를 손실 안에 포함시킨 수치 안정 버전이며, 양성/음성 클래스의 불균형을 보정하기 위해 `pos_weight` 파라미터에 (음성 샘플 수 / 양성 샘플 수) 비율을 넣는다.

추론 단계

학습이 끝난 모델 f_{θ^*}로 새로운 음악을 생성하는 단계를 하나하나 풀어 설명한다.

1단계 — 모델 통과 (logit 생성). 입력 중첩행렬 O_{test}를 모델에 통과시키면 $\hat{z} \in \mathbb{R}^{T \times N}$이 나온다. 이 $\hat{z}$의 각 값은 음수, 0, 양수 모두 가능한 실수이며, 직접 "확률"이라고 해석할 수 없다. 이런 "확률이 되기 전의 raw 점수"를 통계학과 머신러닝에서 **logit**이라고 부른다. 크기가 클수록 "그 note가 활성일 가능성이 높다"는 모델의 내부 점수이다.

2단계 — sigmoid 변환. logit을 0~1 사이의 확률로 바꾸기 위해 sigmoid 함수

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

를 적용한다. sigmoid는 $z=0$에서 0.5, $z \to \infty$에서 1, $z \to -\infty$에서 0에 수렴하는 S자 곡선으로, 실수 전체를 $[0,1]$ 구간으로 눌러 담는다. 적용 후 $P=\sigma(\hat{z}) \in [0,1]^{T \times N}$은 "시점 t에 note n이 활성일 확률"로 해석할 수 있다. 이 단계가 필요한 이유는, 다음 단계에서 "특정 확률 이상인 note를 켠다"는 판단을 내려야 하는데, 그 판단은 반드시 $[0,1]$ 스케일에서 이루어져야 하기 때문이다.

3단계 — adaptive threshold 결정. 가장 단순한 방법은 " $P[t,n] \geq 0.5$이면 켠다"라고 고정 임계값을 쓰는 것이다. 그러나 LSTM이나 Transformer 같은 시퀀스 모델은 학습 결과 sigmoid 출력이 전반적으로 낮게 형성되는 경향이 있어, 0.5를 그대로 쓰면 활성화되는 note가 거의 없어 음악이 텅 비어버린다. 이를 해결하기 위해 본 연구는 원곡의 평균 ON 비율(약 15%)에 맞춰 threshold를 데이터 기반으로 동적 결정한다:

$$\theta = \text{quantile}(P,\ 1-0.15)$$

즉 P의 모든 값 중 상위 15%에 해당하는 경계값을 임계값으로 쓴다. 이렇게 하면 모델 출력의 절대 수준이 어떻든, 전체 활성 note의 비율이 자연스럽게 원곡과 비슷해진다. 이것을 "adaptive threshold"라 부르는 이유는 모델과 입력에 따라 θ 값이 자동으로 달라지기 때문이다.

4단계 — note 활성화 판정. 모든 (t,n) 쌍에 대해 $P[t,n] \geq \theta$이면 시점 t에 note n을 활성화한다. 이 note의 (pitch, duration) 정보를 label 매핑에서 복원하여 $(t,\ \text{pitch},\ t+\text{duration})$ 튜플을 결과 리스트 G에 추가한다.

5단계 — onset gap 후처리. 너무 짧은 간격으로 onset이 연속되면 음악이 지저분해지므로, "이전 onset으로부터 `gap_min` 시점 안에는 새 onset을 허용하지 않는다"는 최소 간격 제약을 적용한다. `gap_min = 0`이면 제약 없음, `gap_min = 3`이면 "3개의 8분음표(= 1.5박) 안에는 새로 타건하지 않음"을 의미한다.

이 과정으로 최종적으로 얻은 $G=[(start, pitch, end), \ldots]$를 MusicXML로 직렬화하면 재생 가능한 음악이 된다.

2.12 두 알고리즘의 비교 요약

항목	Algorithm 1 (Sampling)	Algorithm 2 (DL)
학습 필요 여부	불필요	필요 (E epoch)
결정성	확률적 (난수)	학습 후 결정적
일반화	같은 곡 내부에서만	보지 못한 cycle subset도 생성
위상 보존 방식	교집합 규칙으로 직접 강제	손실함수를 통해 간접

생성 시간	약 50 ms	약 100 ms (학습 후)
학습 시간	해당 없음 (학습 단계 자체가 없음)	30 s ~ 3 min

해석. Algorithm 1은 위상 정보를 직접 규칙으로 강제하므로 cycle 보존도 측면에서 가장 신뢰할 수 있는 기준선 역할을 한다. 반면 Algorithm 2는 학습된 잠재 표현을 통해 부드러운 생성이 가능하며, 학습 데이터에 없는 cycle subset에 대해서도 합리적인 음악을 만들어낸다. 본 연구의 실험에서는 두 알고리즘이 상호 보완적임을 보였다 — Algorithm 1은 위상 보존도에서, Algorithm 2는 음악적 자연스러움에서 각각 우위를 보인다 (Step 4 실험 결과 참조).

Algorithm 1이 "학습 시간" 칸에서 비어 있는 것이 아니라 "해당 없음"으로 표시된 이유는, 이 알고리즘이 애초에 학습 단계를 가지지 않기 때문이다. 주어진 중첩행렬과 cycle 집합만 있으면 어떠한 전처리 학습 없이도 그 자리에서 음악을 생성할 수 있으며, 이것이 Algorithm 1의 가장 큰 장점 중 하나이다.

참고문헌 (Step 2 추가분)

- Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735-1780.
- Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.
- Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.